Introduction to Containerization

- Containerization is the packaging of software code with the operating system (OS) libraries and dependencies required to run the code to create a single lightweight executable—called a container—that runs consistently on any infrastructure.
- More portable and resource-efficient than virtual machines (VMs), containers have become the de facto compute units of modern cloud-native applications.
- Cloud native is an approach to building and running scalable applications to take full advantage of cloud-based services and delivery models.

- A virtual machine (VM) is a virtual representation or emulation of a physical computer that uses software instead of hardware to run programs and deploy applications.
- By using the resources of a single physical machine—memory, CPU, network interface and storage—VMs enable businesses to run multiple machines virtually (with different operating systems) on a single device.
- VMs are typically referred to as guests, with one or more “guest” machines running on a physical machine called the “host” machine.

- https://ubuntu.com/blog/containerization-vs-virtualization
- https://www.ibm.com/think/topics/containerization

Why containers are used by everyone?

- Portability: Buildus once, run anywhere. A container behaves identically on a developer's laptop and a production cluster.
- Speed: Containers start in milliseconds, not minutes, because they don't boot a full operating system.
- Isolation: Each container runs in its own environment. A crash in one container doesn't cascade to others on the same host.
- Scalability: You can spin up hundreds of identical containers in seconds to absorb traffic spikes and scale horizontally.

Virtual Machines vs. Containers

- A Virtual Machine is a digital copy of an entire computer. It uses a hypervisor to carve out physical hardware (CPU, RAM) and installs a complete, heavy “Guest Operating System” on top of it. If you want to run three isolated apps, you need three full operating systems.

- Containerization is a similar but vastly improved concept. Instead of virtualizing the hardware, containers virtualize the operating system. All containers on a machine share the same underlying host OS kernel. This removes the heavy Guest OS layer entirely, making them lighter, faster, and far cheaper to run.

- The concept of containerization and process isolation is decades old.
- The emergence in 2013 of the open-source Docker—an industry standard for containers with simple developer tools and a universal packaging approach—accelerated the adoption of this technology. Today, organizations increasingly use containerization to create new applications and modernize existing applications for the cloud.
- Containers are “lightweight,” meaning they share the machine’s operating system kernel and do not require the overhead of associating an operating system within each application. Containers are inherently smaller in capacity than VMs and require less start-up time. This capability allows far more containers to run on the same compute capacity as a single VM. This capability drives higher server efficiencies and, in turn, reduces server and licensing costs.

- Containerization allows software developers to create and deploy applications faster and more securely, whether the application is a traditional monolith (a single-tiered software application) or a modular application built on microservices architecture.

- Microservices, or microservices architecture, is a cloud-native architectural approach in which a single application is composed of many loosely coupled and independently deployable smaller components or services.

- Containerized applications are run using Docker runtime engine which is installed on the host's operating system and becomes the conduit for containers to share an operating system with other containers on the same computing system.

- **How Docker Works:**
- Docker is the most popular engine used by. container is just a normal Linux process that has been put into isolation using two powerful kernel features to run containers.
- Namespaces (The Blinders): Limits what a process can see. The container gets its own private filesystem, network interface, and process tree, completely hidden from other containers.
- Control Groups / cgroups (The Limits): Limits what a process can use. Ensures the container can only consume a defined amount of CPU or RAM, preventing it from starving other applications on the same host.

- The three core Docker objects used are
- The Dockerfile (a recipe for building your image),
- The Image (an immutable, read-only snapshot),
- The Container (a live, running instance of that image).

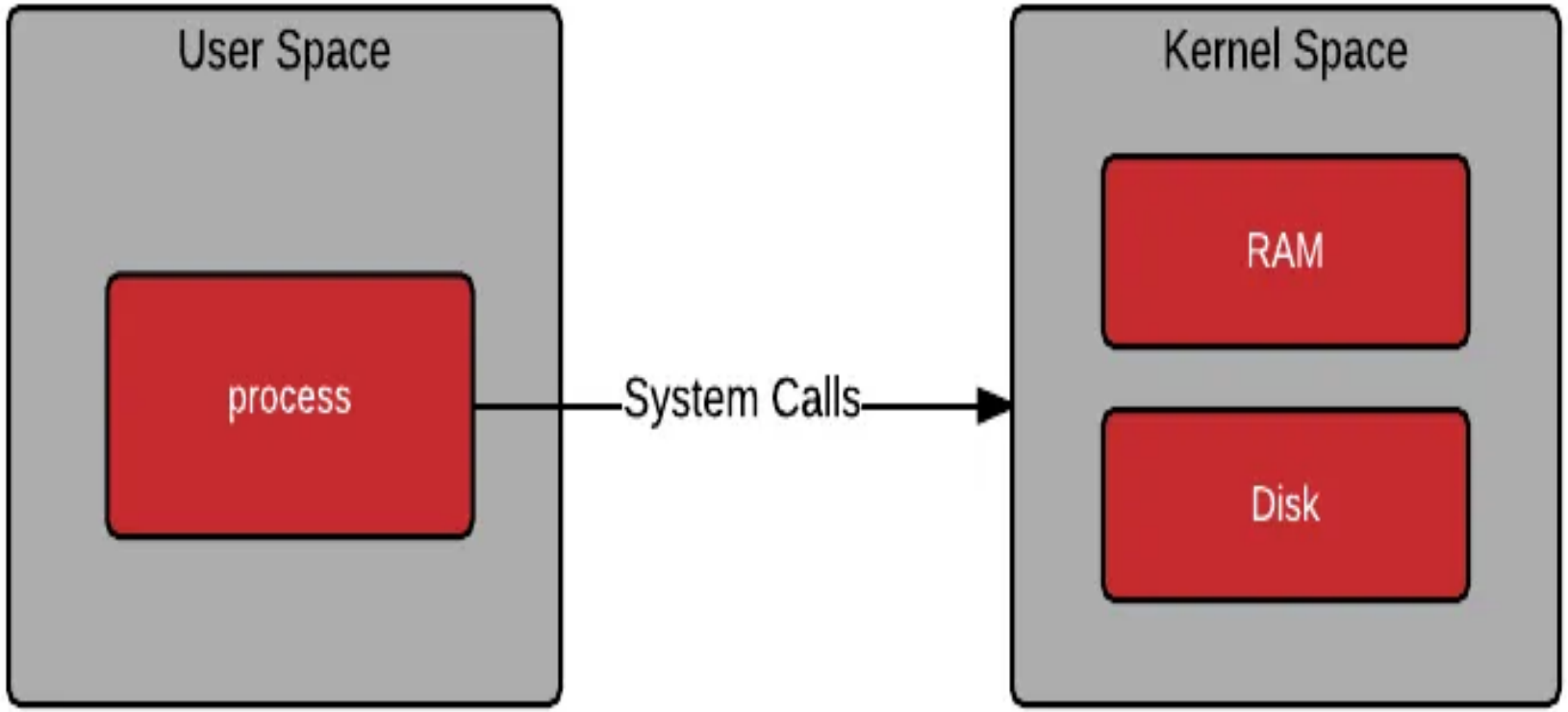
- Docker Example Link

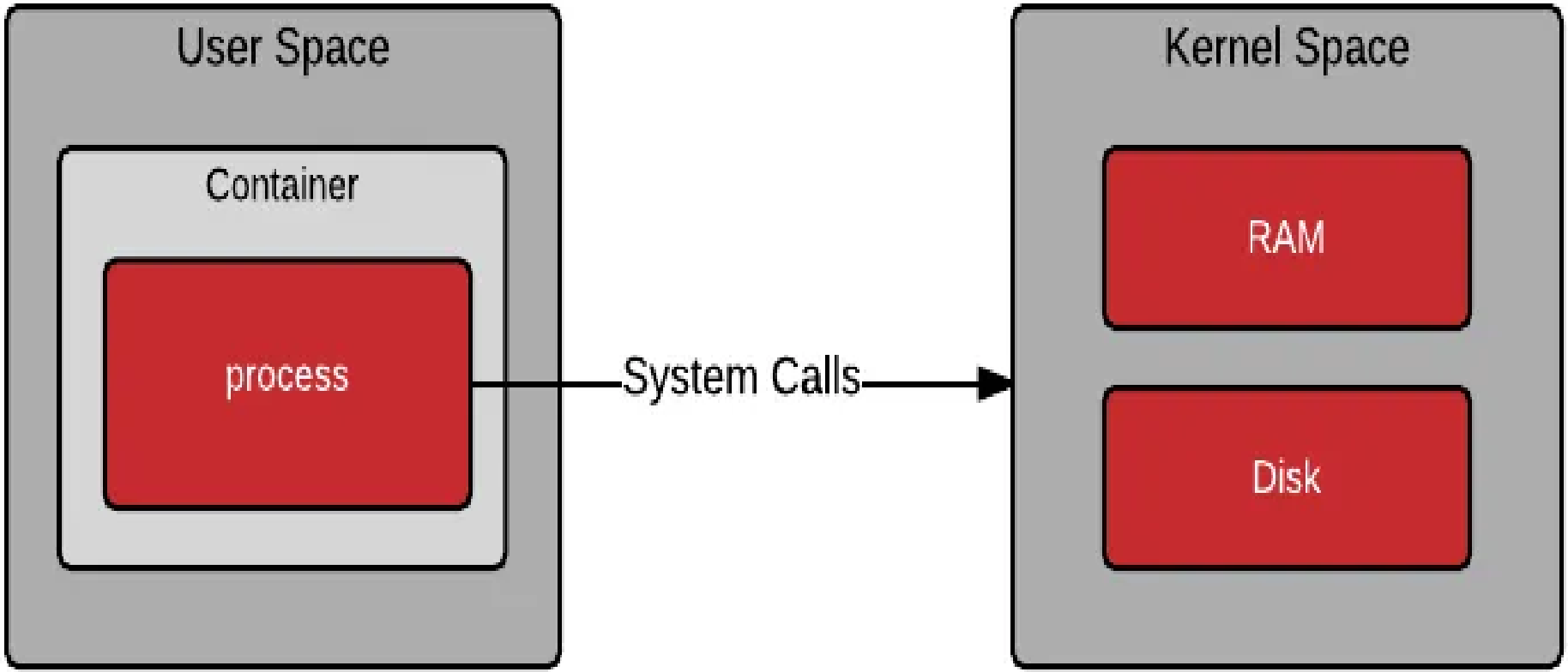
- https://narenm735.github.io/devops/linux/2026/04/09/containerization-docker.html

- User space kernal space
- https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters

Developing a container-based application

- https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters
- The kernel is the only layer of abstraction between programs and the resources they need access to.
- All processes make system calls, As containers are processes, they also make system calls:
-

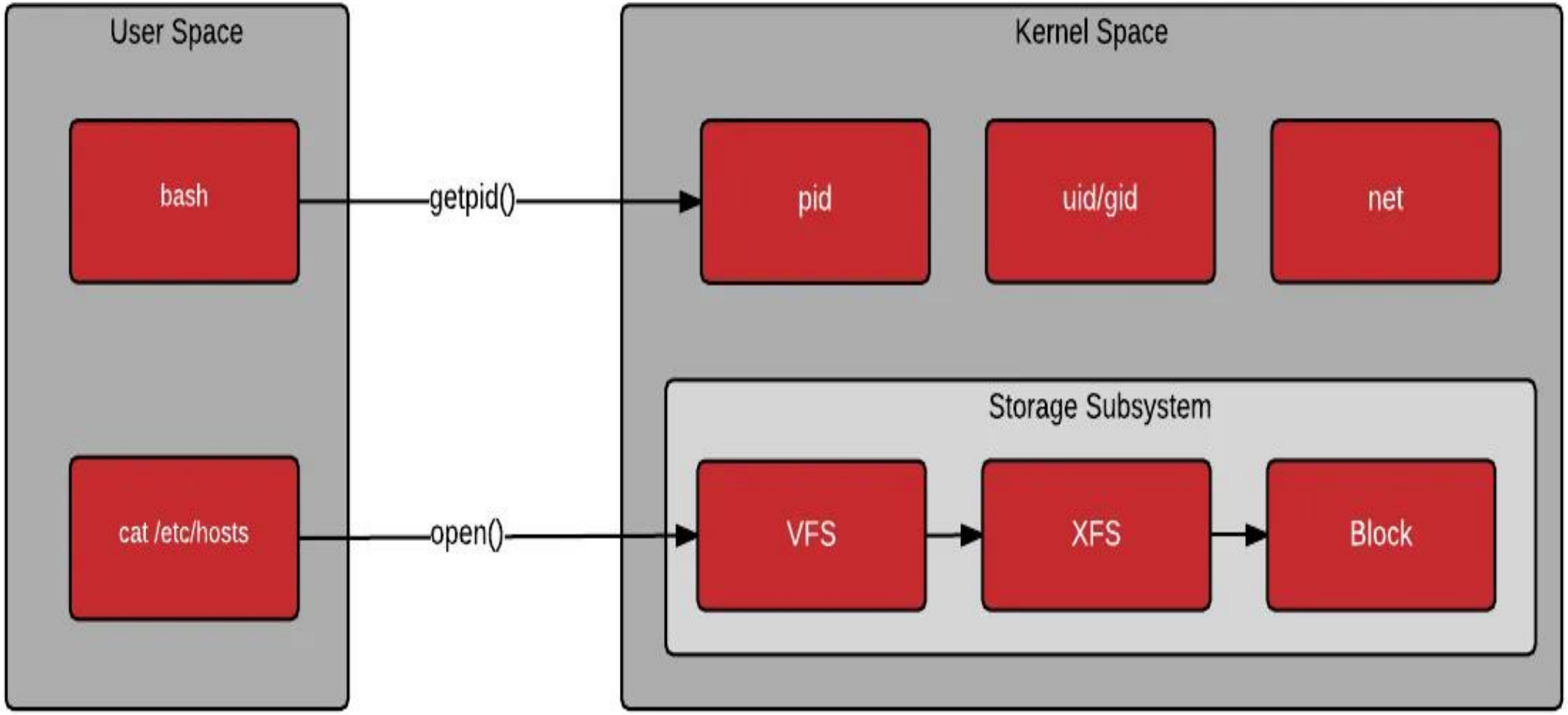

User Space
process
System Calls
Kernel Space
RAM
Disk


User Space
Container
process
System Calls
Kernel Space
RAM
Disk

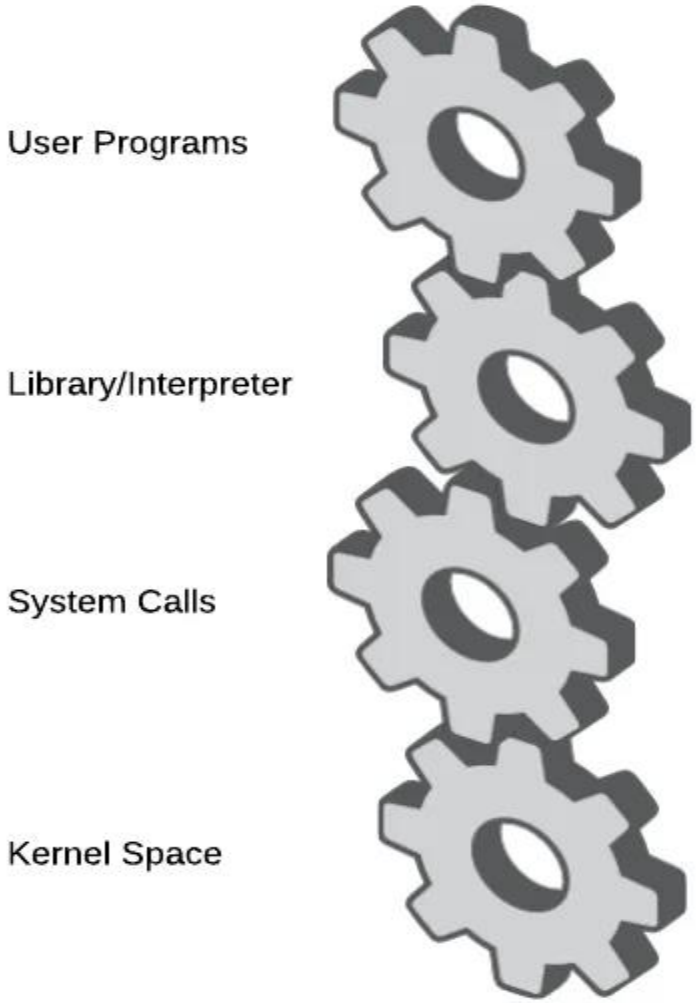
- what about the files and programs that live inside a container image?
- These files and programs make up what is known as user space. When a container is started, a program is loaded into memory from the container image. Once the program in the container is running, it still needs to make system calls into kernel space.
- User Space:
- User space refers to all of the code in an operating system that lives outside of the kernel. Unix-like operating systems come pre-packaged with all kinds of utilities, programming languages, and graphical tools - these are user space applications.
- User applications can include programs that are written in C, Java, Python, Ruby, and other languages. In a containerized world, these programs are typically delivered in a container image format such as Docker.

- When user pulls down and run container image from the Registry, utilizing a pre-packaged, minimal OS user space which contains utilities such as bash, awk, grep, and yum (so that you can install other software).
- docker run -i -t rhel7 bash
- All user programs function by manipulating data, This data can come from registers in the CPU and external devices, but most commonly it is stored in memory and on disk.
- User programs get access to data by making special requests to the kernel called system calls. Examples include allocating memory (variables) or opening a file. Memory and files often store sensitive information owned by different users, so access must be requested from the kernel through system calls.

- Kernel Space
- The kernel provides abstraction for security, hardware, and internal data structures. The open() system call is commonly used to get a file handle in Python, C, Ruby and other languages.
- user programs should not be able to make bit level changes to an file system, so the kernel provides a system call and handles the drivers.
- In the following example *bash* makes a *getpid*() call which requests its own process identity. the *cat* command requests access to */etc/hosts* with a file *open*() call.
- Regular user space programs evoke system calls all the time to get work done, for example: ls ps top bash


User Space
bash
cat /etc/hosts
getpid()
open()
Kernel Space
pid
uid/gid
net
Storage Subsystem
VFS
XFS
Block

- A typical program gets access to resources in the kernel through layers of abstraction similar to the following diagram:



Virtualization:

Organizations often depend on more than one operating system for their daily operations: a Web server on Linux, a mail server on Windows, an e-commerce server for customers running on OS X, and a few other services running on various flavors of UNIX.

A possible (and popular) solution is to use virtual machine technology.

The main idea is that a VMM (Virtual Machine Monitor) creates the illusion of multiple (virtual) machines on the same physical hardware. A VMM is also known as a hypervisor.

Type 1 hypervisors which run on the bare metal, and type 2 hypervisors that may make use of all the services and abstractions offered by an underlying operating system.

Either way, virtualization allows a single computer to host multiple virtual machines, each potentially running a completely different operating system.

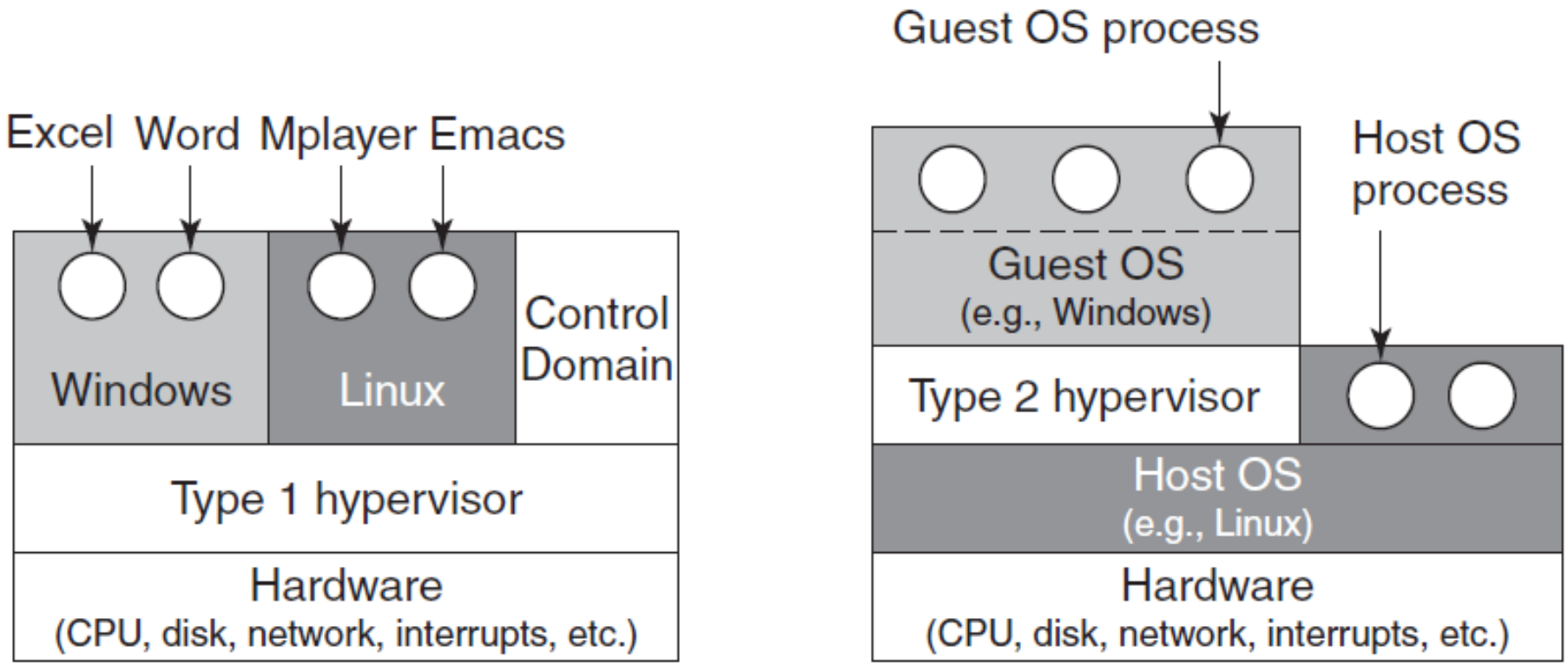
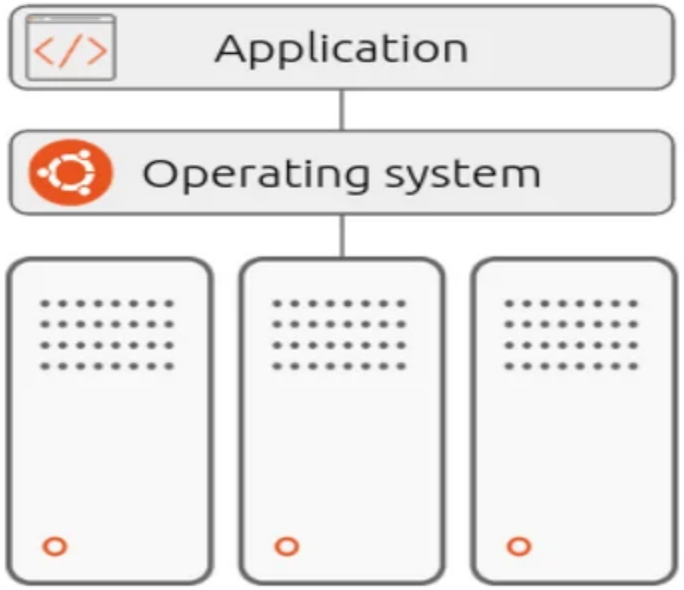


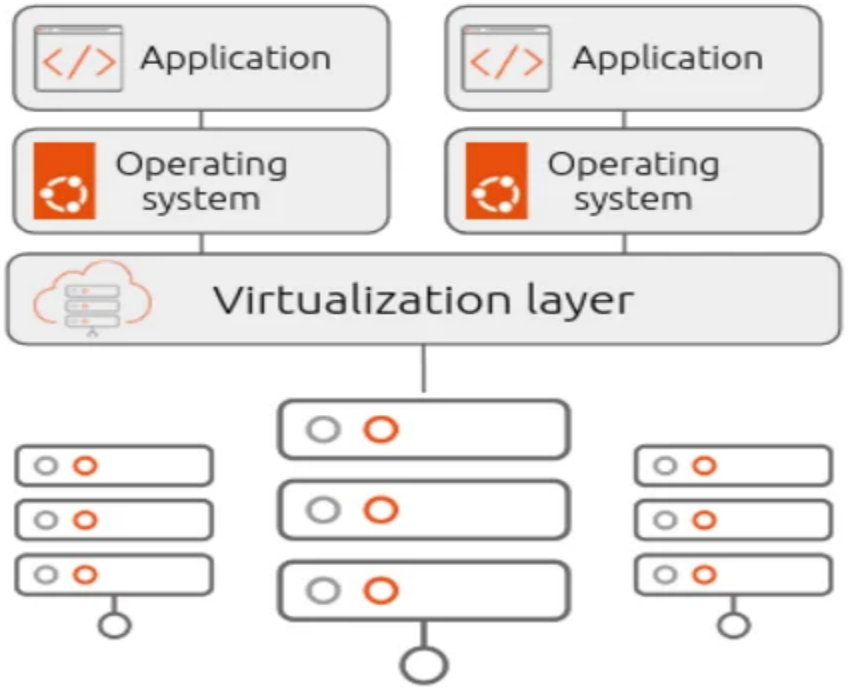
Figure 7-1. Location of type 1 and type 2 hypervisors.

- On a virtualized system, different servers can run on different virtual machines, thus maintaining the partial-failure model that a multicomputer has, but at a lower cost and with easier maintainability.
- With virtual machine technology, the only software running in the highest privilege mode is the hypervisor, which has two orders of magnitude fewer lines of code than a full operating system, and thus two orders of magnitude fewer bugs. A hypervisor is simpler than an operating system because it does only one thing: emulate multiple copies of the bare metal (most commonly the Intel x86 architecture).

- Containers and virtual machines (VMs) are different approaches to packaging computing components and isolating them from the rest of the system. The main difference lies in what components are isolated, which in turn affects the scale and portability of each approach.

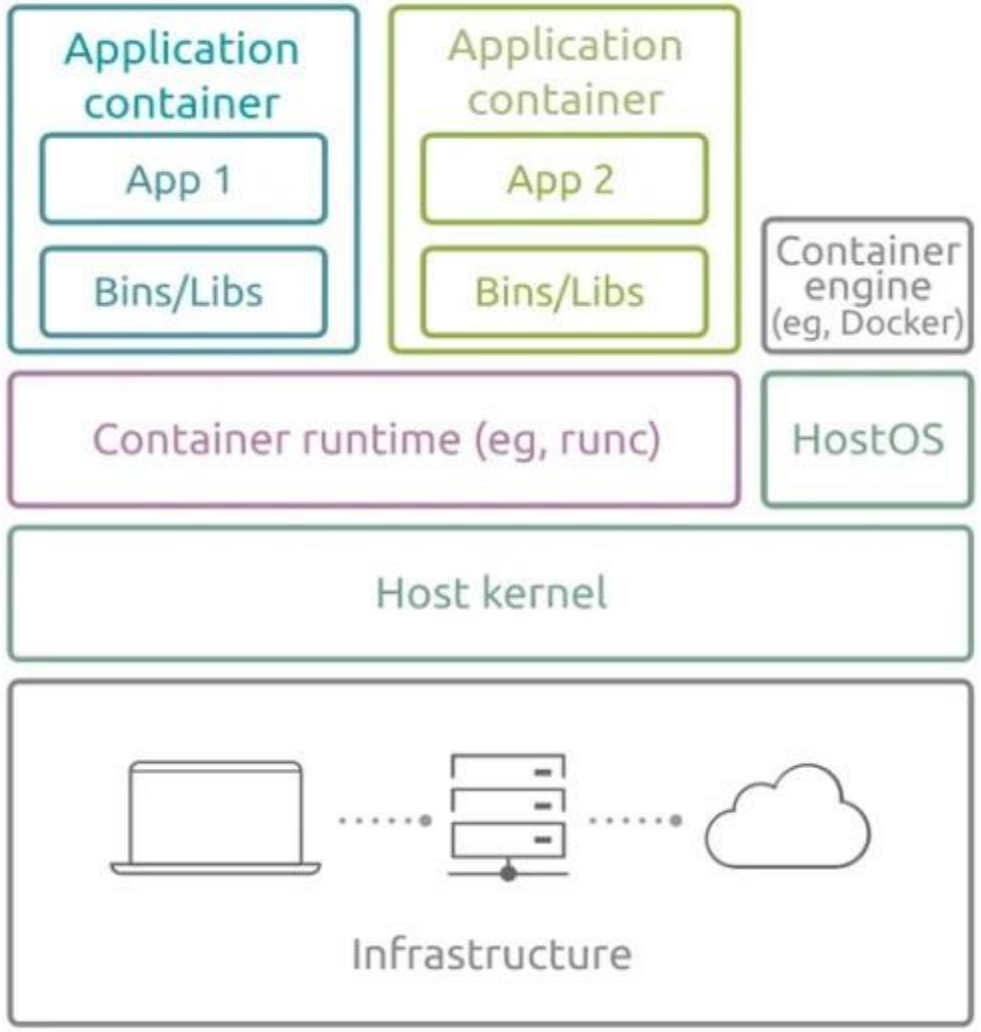


Traditional environment



Virtualized environment

- What is a container?



Application containers (eg, Docker)

- Low-Level Runtime (runc): This is the actual execution engine. It interacts directly with the Linux kernel to create isolated environments using features like namespaces (for isolation) and cgroups (for resource limits). It follows the Open Container Initiative (OCI) standard, which allows it to be swapped for other OCI-compliant runtimes.

- 3 tier microservices architecture:
- Software design that organizes an application into three distinct logical and physical layers: the Presentation Tier, the Application (Logic) Tier, and the Data Tier.
- **Core Structure of the 3 Tiers**
- Presentation Tier (Frontend): The user interface layer where end-users interact with the application. It can include web browsers (using HTML, CSS, JavaScript), mobile apps (using Swift, Kotlin), or desktop GUIs. It gathers user input and displays results but contains no business logic.
- Application Tier (Logic/Microservices): The "brain" of the application where core processing happens. In a microservices model, this tier is composed of many discrete, independently deployable services—such as User, Order, Payment, or Catalog services—that communicate via APIs or messaging protocols.

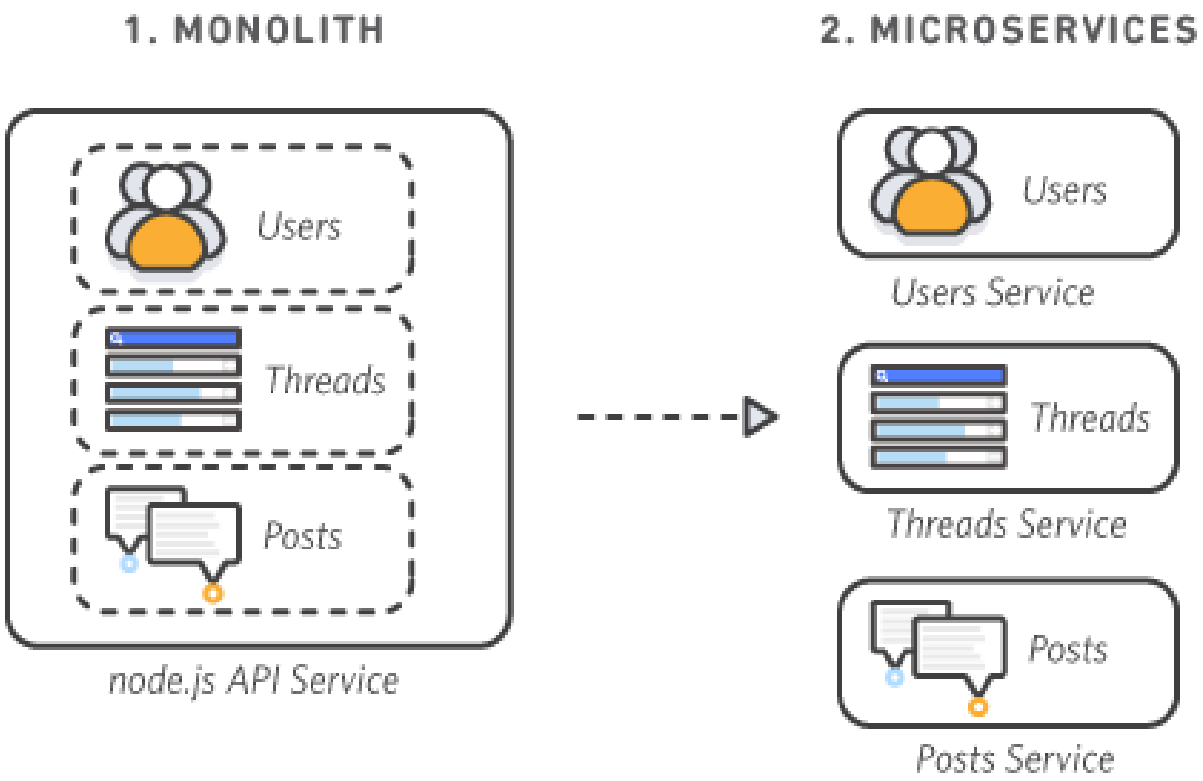
- Data Tier (Database): The storage layer responsible for managing persistent information. Each microservice typically manages its own dedicated database (e.g., MySQL, PostgreSQL, MongoDB, or Redis) to ensure loose coupling and data independence.

- In a typical microservices e-commerce application, the workflow might look like this:
- User Interaction: A customer uses a React-based frontend (Presentation Tier) to browse products.
- Service Coordination: The frontend calls a Catalog Service to find items and a Cart Service to save them (Application Tier).
- Data Persistence: The Catalog Service reads from a PostgreSQL database, while the Cart Service stores temporary data in Redis (Data Tier)

- https://www.ibm.com/think/topics/three-tier-architecture
- What is three-tier architecture?
- Three-tier architecture is a well-established software application architecture that organizes applications into three logical and physical computing tiers: the presentation tier, or user interface; the application tier, where data is processed; and the data tier, where application data is stored and managed.

- https://www.ibm.com/think/topics/microservices?mhsrc=ibmsearch_a&mhq=microservices

- https://aws.amazon.com/microservices



Virtualization

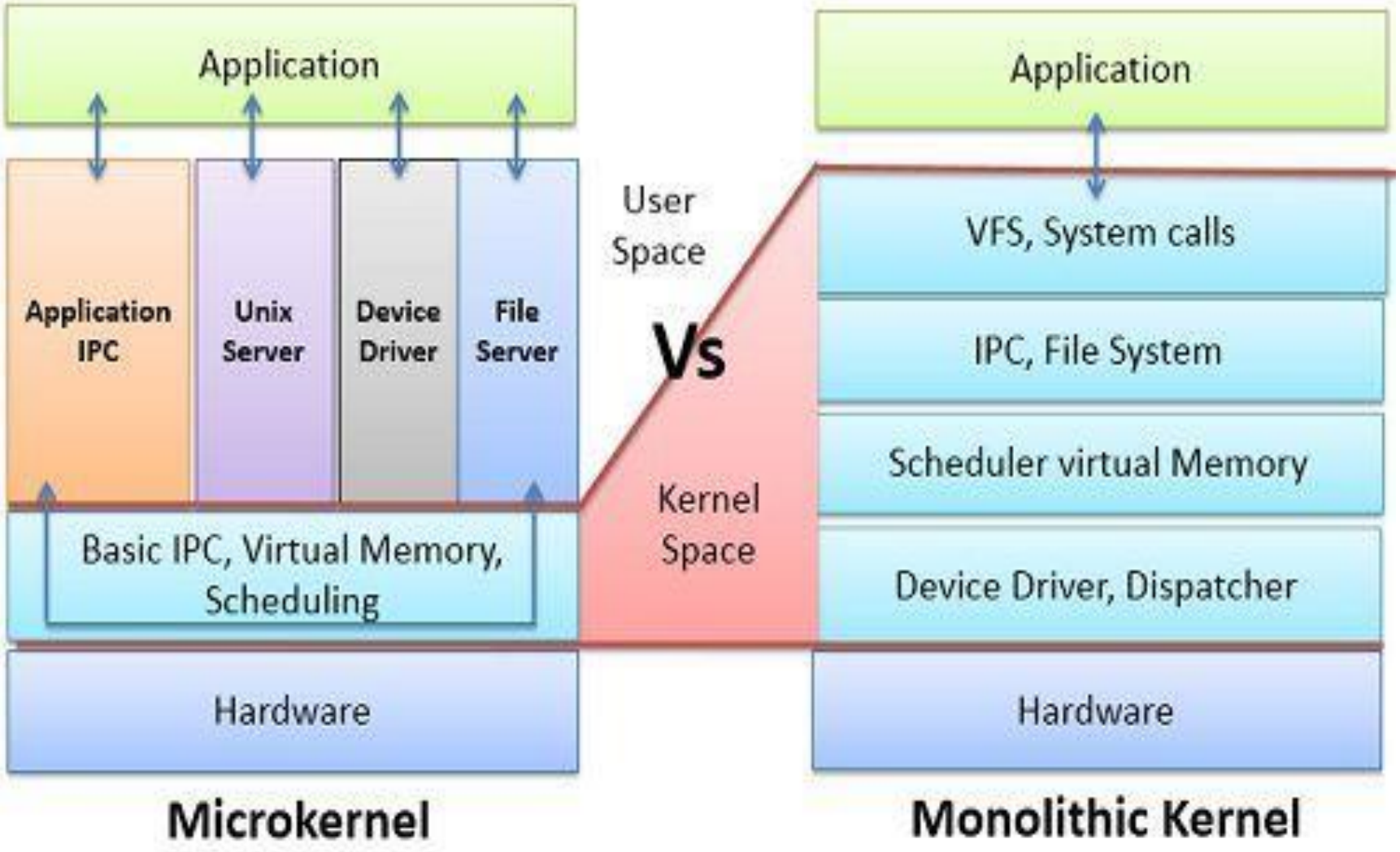
- Organizations often depend on more than one operating system for their daily operations: a Web server on Linux, a mail server on Windows, an e-commerce server for customers running on OS X, and a few other services running on various flavors of UNIX.
- Putting each service on a separate computer, is not a cheap solution.
- A possible (and popular) solution is to use virtual machine technology(introduced in way back 1960s).
- VMM (Virtual Machine Monitor) creates the illusion of multiple (virtual) machines on the same physical hardware. A VMM is also known as a hypervisor.

Microkernel vs. Monolithic Kernel

- Ref:
- https://techdifferences.com/difference-between-microkernel-and-monolithic-kernel.html

- https://www.geeksforgeeks.org/operating-systems/difference-between-microkernel-and-monolithic-kernel/

- Kernel is the core part of an operating system; it manages the system resources.
- Kernel is like a bridge between application and hardware of the computer.
- The Kernel can be classified further into two categories, Microkernel and Monolithic Kernel.
- Microkernel is the one in which user services and kernel services are kept in separate address space.
- Microkernel architecture provides a minimalist operating system design, keeping only essential services—specifically low-level memory management, task scheduling, and inter process communication (IPC)—in kernel mode.


Application
Application IPC
Unix Server
Device Driver
File Server
Basic IPC, Virtual Memory, Scheduling
Hardware
Microkernel
User Space
Vs
Kernel Space
Application
VFS, System calls
IPC, File System
Scheduler virtual Memory
Device Driver, Dispatcher
Hardware
Monolithic Kernel

- Key Microkernel Services (Kernel Space)
- Interprocess Communication (IPC): Crucial for enabling communication between applications and server processes running in user space.
- Low-Level Memory Management: Manages physical memory, address spaces, and mappings.
- Task Scheduling: Handles CPU scheduling algorithms to manage process execution

- **Services Moved to User Space** :

- Device Drivers: Manages peripheral hardware without being in the privileged kernel mode.
- File System: Manages storage, file creation, and permissions, operating independently of the kernel.
- Networking Protocol Stacks: Manages network connectivity, operating in a separate, isolated address space.

- Micro-Kernel:
- A microkernel organizes the operating system by keeping only the most essential functions inside the kernel. These include basic process management, communication between system components, and minimal hardware control. All other services are placed outside the kernel in separate user-space programs.

- Components such as device drivers, file systems, and network services run as independent processes in user space.
- This approach keeps the kernel small, reduces the chance of system-wide failures, and makes the operating system more secure and easier to maintain.

- Advantages of Micro-Kernel
- Size of Micro-Kernel is smaller, so it is easy to use.
- Easy to extend Micro-Kernel
- Easy to port Micro-Kernel
- Micro-Kernel is less prone to errors and bugs. One such example of this is Mac OS.
- Disadvantages of Micro-Kernel
- The execution of Micro-Kernel is slower.
- Only the most important services are present inside the kernel and rest of the operating systems are present inside system application program.
- The communication between client process & services running in user address space is established through message that further reduces the speed of execution.

- Monolithic Kernel
- A monolithic kernel design places all operating system components—both user services and kernel-level services—into the same address space, running entirely in kernel mode. This arrangement allows different parts of the system to interact quickly and efficiently without additional communication overhead.

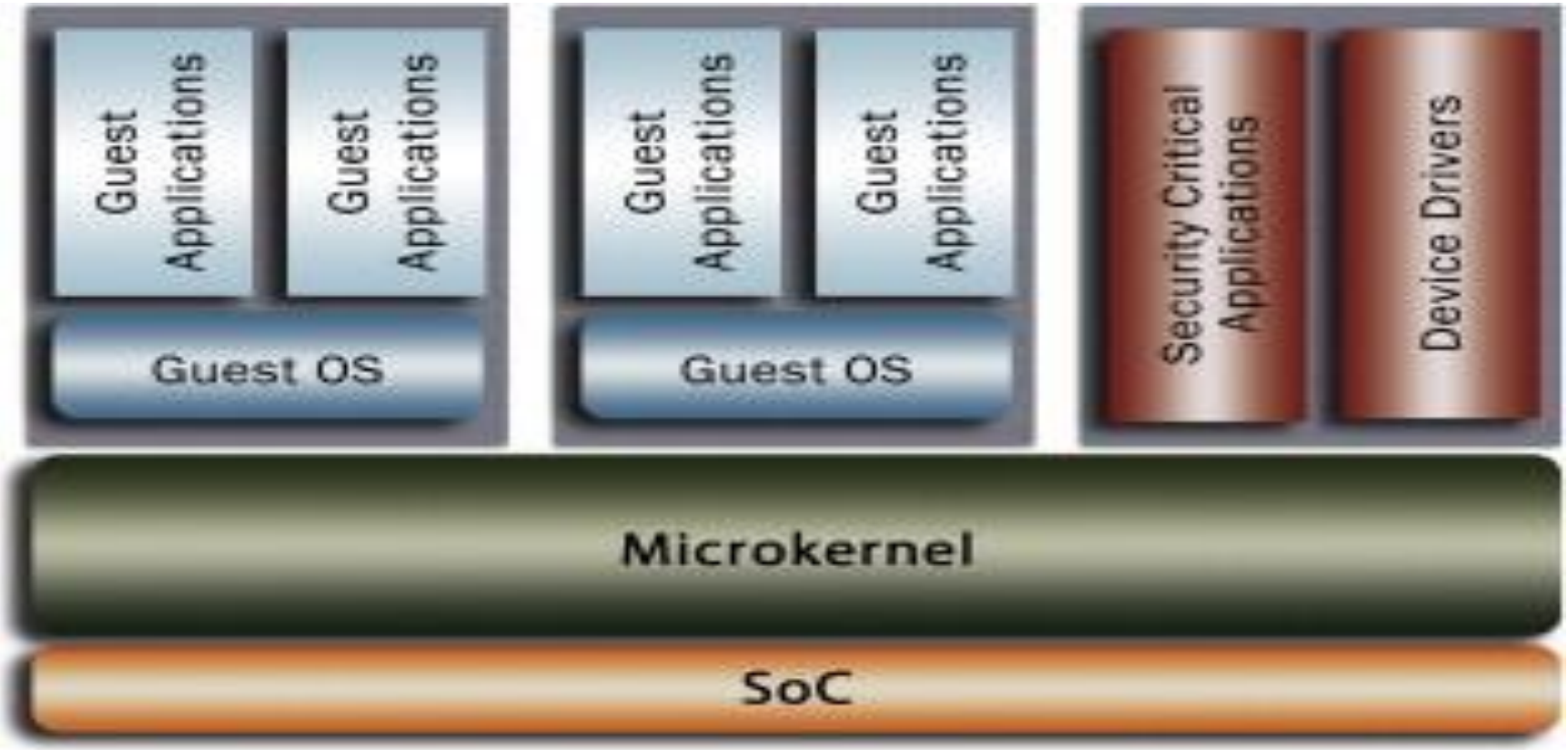
- This architecture includes major functions such as memory management, process control, device drivers, and file systems within one large kernel.
- While this setup can improve performance, it also increases complexity, as a problem in any single component can affect the entire system and make maintenance more challenging.

- Advantages of Monolithic Kernel
- Monolithic Kernel is an all in one piece where user services & kernel services are implemented under the same address space.
- It has a faster speed of execution.
- One such example of this is Linux. It is simple to design and has a very high performance.
- Disadvantages of Monolithic Kernel
- The monolithic kernel has a larger size since both user services & kernel services are implemented under the same space.
- Since it is larger in size, it becomes hard to extend the Monolithic Kernel.
- Hard to export and port the monolithic kernel
- Unlike micro kernel, Monolithic kernel is more prone to errors and bugs. Thus, the system encounters more errors that the usual being.

- Microkernel-based hypervisor:

- Ref:
- Sciencedirect.com/topics/engineering/microkernel
- https://www.sciencedirect.com/topics/engineering/microkernel#:~:text=A%20microkernel%20is%20defined%20as,to%20run%20in%20user%20space.

- The microkernel-based hypervisor, a form of Type-1 architecture, is designed specifically to provide robust separation between guest environments. Because the microkernel is a thin, bare-metal layer, the microkernel-based hypervisor is considered a Type-1 architecture.



- Benefits for Embedded and Safety-Critical Domains
- Automotive: QNX Neutrino runs millions of vehicles and meets ISO 26262 ASIL D safety standards.
- Medical & Industrial Systems: Many microkernels comply with IEC 61508 SIL 3 and Common Criteria certifications.
- IoT & Edge: Lightweight, modular, and remotely updatable environments benefit immensely from microkernel-based OS designs.

- The QNX OS Microkernel
- https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.sys_arch/topic/kernel.html